

Project Proposal

AI-Powered Student Support System

Executive Summary

The proposed solution is an **AI-Powered Student Support System** designed to transform how educational institutions handle student academic inquiries. The institution currently experiences long response times, inconsistent answers, and increased administrative workload due to the overwhelming volume of repetitive student questions related to admissions, course registration, tuition, examinations, graduation requirements, academic calendars, and campus services. To address these challenges, the proposed solution leverages a fully serverless, cloud-native architecture on Amazon Web Services (AWS), integrating modern web technologies, Infrastructure as Code (IaC), containerization, and Generative AI to deliver an intelligent, scalable, secure, and cost-efficient support platform.

The application will provide students with a modern conversational interface where they can ask academic questions and receive immediate AI-generated responses. Every interaction will be securely stored in Amazon DynamoDB, creating an evolving institutional knowledge base that improves response quality while reducing operational costs over time. Administrators will gain valuable insights through dashboards and analytics, enabling them to identify common student concerns, improve institutional documentation, and continuously enhance the quality of student support.

Frontend Technology Evaluation and Selection

Selecting the frontend framework was one of the most important architectural decisions because the application is expected to deliver a fast, responsive, and highly interactive user experience while remaining efficient for users accessing the system over varying network conditions.

Several modern frontend frameworks were evaluated, including **React with Next.js**, **TanStack Start**, and **Qwik**. Next.js is currently one of the most popular React frameworks, offering excellent server-side rendering, routing, and a mature ecosystem. TanStack Start provides outstanding type safety and developer experience, particularly for data-intensive applications. Both frameworks would successfully meet the project's functional requirements.

However, **Qwik with Qwik City** was selected because its architecture aligns exceptionally well with the goals of this project. Unlike traditional React-based frameworks that require client-side hydration before the application becomes interactive, Qwik employs Resumability, allowing the application to render on the server and resume execution in the browser without downloading or executing unnecessary JavaScript during the initial page

load. This significantly reduces startup time, making the application highly responsive even on low-bandwidth mobile networks commonly found in educational environments.

Combined with TypeScript, Tailwind CSS, Qwik UI, Lucide icons, and Framer Motion, the chosen frontend stack enables the development of a polished, responsive, accessible, and visually engaging application while minimizing client-side resource consumption.

See comparison table below

Backend Architecture

The backend will follow a fully serverless architecture built on Amazon Web Services. Amazon API Gateway will expose secure RESTful endpoints that invoke AWS Lambda functions responsible for authentication, conversation management, AI orchestration, analytics, and feedback processing. Amazon Cognito will provide secure user authentication, authorization, and role-based access control for students and administrators. Amazon DynamoDB will serve as the primary NoSQL datastore, storing conversations, cached AI responses, user profiles, feedback, and system audit logs.

To improve scalability and resilience, Amazon SQS will decouple user requests from AI processing, allowing questions to be processed asynchronously during periods of high demand. Amazon SES will be used for email notifications such as account verification, password recovery, and optional conversation summaries, while Amazon SNS will deliver operational alerts generated by CloudWatch Alarms and AWS Budgets to system administrators. Infrastructure provisioning will be fully automated using Terraform, ensuring repeatable, version-controlled deployments across development, testing, and production environments. Docker will standardize local development environments and CI/CD pipelines, guaranteeing consistency between developer workstations and deployment infrastructure.

See comparison table below

Artificial Intelligence Model Evaluation

Choosing the AI platform required balancing three primary considerations: operating cost, response quality, and long-term architectural flexibility.

Several leading AI providers were evaluated, including OpenAI, DeepSeek, Google Gemini, and Amazon Bedrock. OpenAI provides industry-leading conversational quality and an extensive developer ecosystem but requires external API key management, additional secret storage, and tighter integration with third-party services. DeepSeek offers extremely competitive pricing and strong reasoning performance, making it attractive for cost-sensitive deployments; however, its integration into an AWS-first ecosystem is less seamless, and its enterprise tooling is still evolving. Google Gemini provides impressive multimodal

capabilities and competitive pricing but introduces cross-cloud dependencies that increase architectural complexity for an AWS-native solution.

After evaluating these options, Amazon Bedrock was selected as the primary AI platform because it provides the strongest balance between enterprise integration, security, operational simplicity, and long-term maintainability. Rather than exposing API keys, Bedrock integrates directly with AWS Identity and Access Management (IAM), allowing Lambda functions to securely invoke foundation models using role-based permissions. Bedrock also provides access to multiple leading foundation models—including Anthropic Claude, Amazon Nova, Meta Llama, and Mistral—through a single managed service, allowing the application to adopt the most appropriate model for each workload without changing the overall system architecture.

See comparison table below

Avoiding Tight Coupling

Although Amazon Bedrock is the preferred AI platform, the solution is intentionally designed to avoid vendor lock-in. Rather than embedding Bedrock-specific logic throughout the application, an abstraction layer will be introduced through an **AI Provider Interface**. All business logic will communicate with a generic `AIProvider` interface responsible for generating responses, retrieving embeddings, and managing conversations. Concrete implementations such as `BedrockProvider`, `OpenAIProvider`, `DeepSeekProvider`, or future providers can be introduced without modifying the application's business logic. This architectural pattern promotes loose coupling, simplifies testing, and allows the institution to adopt new models as pricing, performance, or organizational requirements evolve. Should another provider become more cost-effective in the future, the backend can transition with minimal changes while preserving the existing API contracts and frontend experience.

Cost Optimization Strategy

Cost efficiency is a central design principle of the proposed solution. Rather than invoking a large language model for every student query, the system follows a multi-layered decision process that minimizes unnecessary AI requests. Incoming questions are first compared against previously answered questions stored in DynamoDB. If a suitable answer already exists, the cached response is returned immediately, eliminating additional inference costs. If no cached response is available, the system attempts to retrieve relevant institutional information through a retrieval mechanism backed by institutional documentation. Only when sufficient confidence cannot be achieved does the application invoke Amazon Bedrock to generate a new response. Newly generated responses are subsequently stored for reuse, progressively reducing AI inference costs as the knowledge base expands.

The architecture also supports intelligent model routing, where lower-cost models such as Amazon Nova Lite can handle routine administrative questions, while more capable models

such as Claude Sonnet are reserved for complex academic reasoning. Conversation summaries will be maintained to reduce prompt size and token consumption, while AWS Budgets, CloudWatch, and SNS will continuously monitor infrastructure usage and notify administrators of unusual spending patterns. Together, these optimizations significantly reduce operational expenses while maintaining high-quality responses.

See comparison table below

Security and Scalability

Security is implemented following AWS best practices. Amazon Cognito provides secure authentication with JWT-based authorization, while IAM enforces least-privilege access across all services. API Gateway protects backend endpoints, and no AI credentials are ever exposed to the client. Sensitive configuration values are securely managed through AWS-native mechanisms, and HTTPS encryption protects all communication between users and backend services.

The serverless architecture ensures that compute resources scale automatically based on demand without requiring manual infrastructure management. AWS Lambda scales independently for each function, DynamoDB On-Demand adapts to fluctuating workloads, SQS absorbs traffic spikes, and CloudFront accelerates content delivery globally. Terraform guarantees consistent infrastructure provisioning, while Docker standardizes development and deployment environments, simplifying collaboration and reducing configuration drift.

Conclusion

The proposed AI-Powered Student Support System combines modern frontend engineering, serverless cloud architecture, infrastructure automation, and enterprise-grade artificial intelligence into a cohesive, production-ready solution. By selecting **Qwik** for its resumability and exceptional performance characteristics, the application delivers an outstanding user experience with minimal client-side overhead. By adopting **Amazon Bedrock** as the primary AI platform while introducing an abstraction layer to avoid tight coupling, the solution achieves a balance between high-quality AI capabilities, AWS-native integration, and long-term architectural flexibility. Through intelligent caching, retrieval-based augmentation, asynchronous processing, and model routing, the system minimizes operational costs without compromising response quality. The result is a scalable, secure, maintainable, and future-ready platform that not only addresses the institution's current support challenges but also establishes a strong foundation for future AI-driven educational services.

COMPARISON TABLES

Table 1: Frontend Framework Comparison

Criteria	Qwik City	Next.js 15	TanStack Start
Performance	Excellent (Resumability, near-zero hydration)	Excellent (SSR + React Server Components)	Excellent
Initial Load Time	Extremely Fast	Fast	Fast
JavaScript Sent to Browser	Minimal (loads only when needed)	Moderate	Moderate
Hydration Required	No (Resumability)	Yes	Yes
Server-Side Rendering	Yes	Yes	Yes
Progressive Enhancement	Excellent (Native HTML forms work without JavaScript)	Good	Good
SEO Support	Excellent	Excellent	Very Good
Developer Experience	Very Good	Excellent	Excellent
Learning Curve	Steeper	Moderate	Moderate
Ecosystem	Growing	Largest React ecosystem	Growing rapidly
TypeScript Support	Excellent	Excellent	Excellent
Tailwind CSS Support	Excellent	Excellent	Excellent
AWS Integration	Excellent	Excellent	Excellent
Best Use Case	Performance-first, server-first applications	General-purpose React applications	Data-driven applications

Why Qwik Was Selected

Decision Factor	Justification
Performance	Qwik's resumability eliminates traditional hydration, resulting in exceptionally fast page loads and reduced JavaScript execution.
Student Experience	Many students access university portals using low-bandwidth mobile networks. Qwik provides a smoother experience under these conditions.
Security	Qwik City's server actions allow sensitive operations to remain entirely on the server, preventing exposure of backend logic or AI credentials.
Modern Architecture	Demonstrates familiarity with emerging frontend technologies beyond mainstream React frameworks.
Portfolio Value	Differentiates the project by showcasing a less common but highly innovative framework.

Table 2: AI Model Comparison

Criteria	Amazon Bedrock (Selected)	OpenAI	DeepSeek	Google Gemini	Self-hosted Llama
Cost	Very Good	Good	Excellent	Very Good	Excellent (after infrastructure)
Response Quality	Excellent	Excellent	Very Good	Very Good	Very Good
AWS Integration	Excellent	Fair	Fair	Fair	Very Good
IAM Authentication	Yes	No (API Keys)	No (API Keys)	No (API Keys)	Depends
Multiple Models	Yes	No	No	No	Limited
Vendor Flexibility	Excellent	Good	Good	Good	Very Good
Security	Excellent	Excellent	Good	Excellent	Depends on implementation
Guardrails	Built-in	Limited	Limited	Limited	Custom
Knowledge Base Support	Native	Custom Implementation	Custom Implementation	Custom Implementation	Custom Implementation
Enterprise Readiness	Excellent	Excellent	Good	Very Good	Good
AWS Portfolio Value	Excellent	Good	Fair	Fair	Very Good

Why Amazon Bedrock Was Selected

Decision Factor	Justification
AWS Native	Integrates seamlessly with IAM, Lambda, CloudWatch, Terraform, and the broader AWS ecosystem.
Security	No API keys are exposed or managed. Lambda functions authenticate using IAM roles, reducing operational risk.
Model Flexibility	A single API provides access to multiple foundation models, including Anthropic Claude, Amazon Nova, Meta Llama, Mistral, Cohere, and others.
Future-Proofing	New models can be adopted without redesigning the application's architecture or changing frontend integrations.

Decision Factor	Justification
Operational Simplicity	Centralized monitoring, permissions, and governance remain within AWS.
Portfolio Strength	Demonstrates practical experience with AWS's managed Generative AI platform, an increasingly valuable enterprise skill.

Table 3: AI Provider Trade-offs

Provider	Strengths	Weaknesses
Amazon Bedrock	AWS-native, IAM authentication, multiple models, Guardrails, Knowledge Bases, enterprise integration	Slightly higher learning curve, some models may cost more than direct provider APIs
OpenAI	Outstanding reasoning, mature ecosystem, excellent documentation	Requires API keys, external dependency, vendor-specific APIs
DeepSeek	Extremely cost-effective, strong reasoning, fast inference	Less mature enterprise tooling, not tightly integrated with AWS
Google Gemini	Strong multimodal capabilities, competitive performance	Cross-cloud dependency for AWS deployments
Self-hosted Llama	Maximum control, no vendor dependency, customizable	Requires infrastructure management, scaling, monitoring, updates, and security hardening

Table 4: Cost Comparison

Feature	Amazon Bedrock	OpenAI	DeepSeek	Winner
Infrastructure Management	Excellent	Very Good	Very Good	Bedrock
Authentication	IAM Roles	API Keys	API Keys	Bedrock
Operational Overhead	Low	Medium	Medium	Bedrock
Entry Cost	Medium	Medium	Low	DeepSeek
Long-term AWS Integration	Excellent	Moderate	Moderate	Bedrock
Multi-model Switching	Excellent	Limited	Limited	Bedrock
Best Overall Enterprise Value	Excellent	Very Good	Very Good	Bedrock

Expected Benefits

The proposed solution delivers significant benefits across multiple dimensions.

Category	Benefits
Students	Instant AI-powered academic assistance, faster response times, improved user experience, 24/7 availability
Administrative Staff	Reduced workload, fewer repetitive inquiries, improved operational efficiency
Institution	Lower operational costs, scalable cloud infrastructure, centralized knowledge repository, actionable analytics
Technical Team	Infrastructure as Code, automated deployments, containerized development, maintainable architecture
Portfolio Value	Demonstrates expertise in AWS Serverless, Terraform, Docker, Qwik, Amazon Bedrock, CI/CD, and modern cloud-native AI engineering

Conclusion

The proposed AI-Powered Student Support System combines cutting-edge frontend engineering, serverless cloud architecture, infrastructure automation, and enterprise-grade Generative AI into a unified, production-ready solution. By selecting Qwik for its resumability and exceptional performance characteristics, the application delivers a fast, responsive, and accessible user experience with minimal client-side overhead. Amazon Bedrock was chosen as the primary AI platform because it offers seamless AWS integration, IAM-based security, access to multiple foundation models, and native support for enterprise AI capabilities such as Guardrails and Knowledge Bases. To ensure long-term flexibility and avoid vendor lock-in, the solution introduces an AI Provider Interface, enabling alternative AI providers to be adopted with minimal architectural changes. Combined with intelligent caching, retrieval-based augmentation, asynchronous processing, model routing, Terraform-based infrastructure provisioning, Dockerized development environments, and automated CI/CD pipelines, the proposed system delivers a secure, scalable, cost-efficient, and future-ready platform capable of transforming student support services while showcasing modern cloud engineering and AI best practices.